

RESEARCH ARTICLE

Federated Reinforcement Learning in Stock Trading Execution: The FPPO Algorithm for Information Security

HAOGANG FENG¹, YUE WANG², SHIDA ZHONG^{1,2}, TAO YUAN¹, (Member, IEEE),
AND ZHI QUAN¹, (Senior Member, IEEE)

¹State Key Laboratory of Radio Frequency Heterogeneous Integration, Shenzhen University, Shenzhen 518060, China

²College of Electronics and Information Engineering, Shenzhen University, Shenzhen 518060, China

Corresponding author: Shida Zhong (shida.zhong@szu.edu.cn)

This work was supported in part by the National Key Research and Development Program, China, under Project 2023YFB4403805; and in part by the (Key Area) Project of Department of Education of Guangdong Province, China, under Project 2021ZDZX4008.

ABSTRACT Stock trading execution is a critical component in the complex financial market landscape, and the development of a robust trade execution framework is essential for financial institutions pursuing profitability. This paper presents the Federated Proximal Policy Optimization (FPPO) algorithm, an adaptive trade execution framework that leverages joint reinforcement learning. The FPPO algorithm demonstrates significant improvements in model performance across various stocks, with average returns enhanced by 3% to 15%. It also exhibits superior performance in key metrics such as the reward function value, showcasing its effectiveness in different financial contexts. The paper further explores the model's performance under the FPPO algorithm with varying numbers of client nodes and different risk preferences, underscoring the importance of these factors in model construction. The results substantiate the FPPO algorithm's capability to safeguard privacy, ensure high performance, and enable the creation of personalized trading models in the optimal trade execution problem. This positions investors to gain a competitive edge in the dynamic and complex financial markets. Although the FPPO algorithm demonstrates significant potential in trade execution optimization, it may need to integrate a broader range of real-world variables and develop advanced privacy-preserving mechanisms to enhance its applicability in diverse financial contexts.

INDEX TERMS Federated reinforcement learning, optimal execution, privacy protection.

I. INTRODUCTION

In the field of financial markets, stock trading execution plays a crucial role in achieving efficient and cost-effective trading outcomes. Traditionally, trade execution strategies have relied on predefined rules and algorithms that may not adapt well to dynamic market conditions. Optimal execution is aimed at ensuring that clients receive the most favorable conditions in trading, with quantitative trading employing mathematical models for trading decisions, and financial technology enhancing financial service efficiency through technological innovation. A diverse array of traditional methodologies are employed in crafting stock trading

models. These approaches typically leverage historical price data, market indicators, technical metrics, and fundamental analysis to forecast future stock price movements. Furthermore, time series techniques like moving averages and auto-regressive moving average models (ARMA) are harnessed to develop optimal trade execution systems [1]. Additionally, scholars often harness machine learning algorithms such as convolutional neural networks (CNNs) [2] and decision trees [3] to discern patterns and make predictions using vast historical datasets. However, the intricate and unpredictable nature of the stock market presents a challenge. None of these strategies possess the capability to dynamically adjust the model in real-time to accommodate the ever-changing market conditions. Consequently, achieving satisfactory modeling outcomes in the financial realm,

The associate editor coordinating the review of this manuscript and approving it for publication was Gustavo Olague¹.

where the pace of change is exceedingly rapid, proves to be a daunting task.

To surmount this limitation, researchers have turned their attention to reinforcement learning (RL) techniques, which empower adaptive and data-driven trading strategies. RL [4] endeavors to learn optimal policies for sequential decision problems by optimizing a cumulative future reward function, with minimal modeling assumptions (such as the Markovian structure of the state space). In tackling the optimal execution problem, researchers have deployed various RL techniques, including temporal-difference learning, policy gradient, and reinforce [5]. Among these, Q-learning emerges as the most popular RL algorithm [6], notably applied to optimal execution challenges in finance [7], [8]. Lin and Beling [9] presents a pioneering deep reinforcement learning (DRL) framework that leverages an advanced variant of the Deep Q-Network (DQN) algorithm, incorporating Double DQN, Dueling Network, and Noisy Nets to minimize trade execution costs. Chen et al. [10] presents a novel approach termed Deep Dyna-Double Q-learning (D3Q), which integrates deep reinforcement learning with planning to enhance trading performance while significantly reducing the training overhead associated with direct reinforcement learning methods. Kim et al. [11] propose a Transformer model to capture the overall distribution of daily volumes in a U-shape and an LSTM model to handle the order distribution within smaller time intervals, using Proximal Policy Optimization (PPO) as the learning algorithm. Moallemi and Wang [12] explores the application of reinforcement learning to the problem of optimal trade execution, providing a novel framework that dynamically determines the timing and size of trades to minimize market impact and execution costs, thereby enhancing traditional algorithms and offering a more adaptive and efficient strategy for high-frequency trading environments. Ansari et al. [13] proposes a decision support system based on deep reinforcement learning that helps trading agents make optimal decisions to buy, sell or hold stocks by combining past and future stock price trends as observation states. Bisht and Kumar [14] proposes a multi-objective system based on reinforcement learning. The system not only focuses on maximizing returns, but also considers risk control to obtain maximum profit while adjusting risk through a multi-objective optimization approach. In addition, reinforcement learning has showcased promising outcomes across diverse domains, spanning games, robotics, and finance [5], [15], [16]. RL algorithms learn through interactions with an environment, wherein an agent undertakes actions to maximize cumulative rewards over time. In the context of trade execution, RL agents are trained to make optimal decisions concerning trade sizes, timing, and order placement, all aimed at minimizing costs and market impact. However, the application of RL to trade execution confronts challenges linked to data privacy and scalability. Market participants often exhibit reluctance to divulge their proprietary trading information, citing concerns

related to privacy protection and maintaining a competitive edge.

Federated Learning (FL) stands out as a distributed learning approach facilitating model update sharing across devices without exposing raw data, thus safeguarding users' personal information from potential threats. It addresses privacy leakage risks inherent in model training parameters, offering enhanced privacy. Moreover, conventional methods like Differential Privacy (DP), Local Differentially Private Federated Learning (LDP-Fed), and Global Differentially Private Federated Learning (GDP-FL) also contribute to data protection, albeit with varying degrees of effectiveness. Compared to DP, LDP-Fed, and GDP-FL, Federated Learning boasts a significant advantage: it can maintain high classification accuracy while tailoring privacy protection to individual needs. For instance, on datasets like MNIST and CIFAR-10, a Federated Learning variant called PDP-FL [17] achieved classification accuracies of 93.8% to 94.5% and 43.4% to 45.2%, respectively, outperforming LDP-Fed and GDP-FL in both local and global differential privacy settings. This demonstrates Federated Learning's superior balance between data protection efficacy and model performance. FL has already demonstrated impressive results across various domains, including fraud detection [18] and medical modeling [19]. However, despite its potential in addressing data privacy and security concerns, limited research has explored its application in constructing stock trading models.

In summary, stock trade execution faces three critical challenges: (1) the necessity for rapid and dynamic adaptation to unpredictable and ever-changing market conditions; (2) the requirement to ensure robust privacy protection while meeting the diverse customization needs of market participants; and (3) the demand for unique execution strategies tailored to specific stocks or market environments, as their distinct characteristics necessitate specialized approaches. To address these challenges, federated reinforcement learning (FRL) [20] has emerged as a promising solution. FRL integrates the advantages of RL with the decentralized approach of federated learning, enabling multiple market participants to jointly train a global RL model while preserving the privacy of their local data. Federated learning enables multiple financial institutions or agents to share the learning outcomes of a model while preserving the privacy and security of their sensitive local data. Each participant can build personalized trading strategies by combining the shared global model with their own local data. This mechanism is particularly crucial in financial markets, where different stocks or market environments often require distinct execution strategies. Reinforcement learning, on the other hand, optimizes trading actions through iterative improvement based on experiential data, making it highly effective in multi-agent scenarios. The integration of federated learning and reinforcement learning not only ensures data security and privacy but also enhances the model's trading capabilities. This collaborative approach improves the efficiency and adaptability of

trade execution, supports the creation of personalized strategies for varying market conditions, and results in a more robust and effective trading framework. Lin and Beling [21] proposed an innovative end-to-end adaptive framework for optimal trade execution using Proximal Policy Optimization (PPO) and incorporating market time dependencies through Long Short-Term Memory (LSTM) or Fully Convolutional Network (FCN) architectures. This framework outperformed industry baselines. To further optimize the convergence time and training loss of federated learning, Sharma and Bhadula [22] explored the critical role of secure federated learning in smart industrial IoT-enabled applications, highlighting its ability to protect the privacy of sensitive personal data while decentralizing the training of machine learning models. Liang et al. [23] introduces an innovative online federated reinforcement learning transfer process that enables real-time knowledge extraction among participant agents, even when they operate in different environments. The key contribution is the demonstration of knowledge transfer from simulator car agents to real-world RC cars during collision avoidance tasks, showcasing the practical application of federated transfer reinforcement learning in enhancing autonomous driving systems. Despite the progress made in the aforementioned areas of Federated Reinforcement Learning, there are still issues that need to be improved. In particular, in terms of privacy protection and model customization, more refined strategies are needed to ensure data security and meet the specific needs of different domains. In addition, in the area of dynamic environment modeling and model performance optimization, further research is needed on how to make models better adapt to environmental changes and improve model stability and efficiency.

In this paper, the construction of optimal trading models using federated learning is employed to provide more efficient and secure trading strategies for financial markets. The limitations of traditional centralized RL methods are addressed by utilizing FRL, which enables market participants to collectively learn from their local data while maintaining data privacy. The decentralized nature of FRL allows for the exploration of diverse trading strategies and the aggregation of knowledge from multiple participants, ultimately leading to improved execution outcomes and enhanced market efficiency. Our main contributions can be summarized in three aspects: 1. An optimal trade execution framework based on FPPO is proposed. This framework takes into account temporal correlation and enables decision-making based on raw secondary market micro-structure data, rather than relying on manually designed attributes. The privacy of all parties is secured through federated learning, enabling the training of high-performance privacy-preserving trading models. 2. A reward function incorporating risk preferences is introduced, allowing different participants to customize their trading models according to their risk tolerance levels. 3. Extensive experiments are conducted to demonstrate the advantages of our framework over several algorithms commonly used in the financial industry, including TWAP,

AC, and DQN models. Our results show that our proposed framework outperforms these existing methods in terms of various performance metrics.

The subsequent sections of this paper are organized as follows: In Section II, we provide a detailed exposition of the optimal execution problem within a reinforcement learning framework, along with its adaptation for federated learning. Section III elucidates the network training process, presenting a comprehensive algorithm. The experimental methodology and analysis of evaluation results are delineated in Section IV. Lastly, Section 5 offers concluding remarks on the study and outlines avenues for future research.

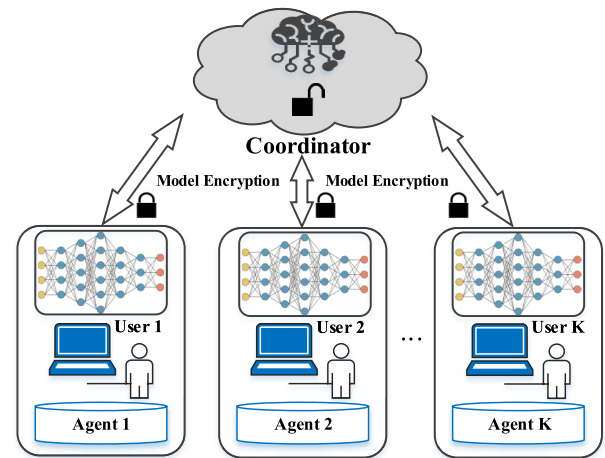


FIGURE 1. FPPO model.

II. SYSTEM DESIGN AND MODELING

As illustrated in Fig. 1, an overview of the FPPO algorithm-based approach for constructing privacy-preserving optimal trading execution is provided. The global model is initialized on the federated server and then distributed to local agents for training and reinforcement learning. The trained models are then encrypted and transmitted to the federated server for aggregation. Afterwards, the aggregated model is distributed back to the local models for deployment and subsequent execution of trading with the well-constructed and high-performing model.

A. PRELIMINARIES

Optimal Execution: The optimal execution problem revolves around a trader who, at $t = 0$, holds an inventory of q_0 shares and must fully liquidate it by the end of the time period $t = T$ (i.e., $q_T = 0$). This problem is analyzed in discrete time intervals, where trade orders $x_t = 0$ are executed at evenly distributed time steps $t = 0, 1, 2, \dots, T - 1$, with x_t denoting the number of shares to be sold at each step. The midprice p_t follows an unknown process, which may or may not be influenced by the trader's actions.

As shown in Fig. 2, it is possible to model the process as a Markov decision process (MDP). Here, the agent interacts with the environment, represented by the stock market, based

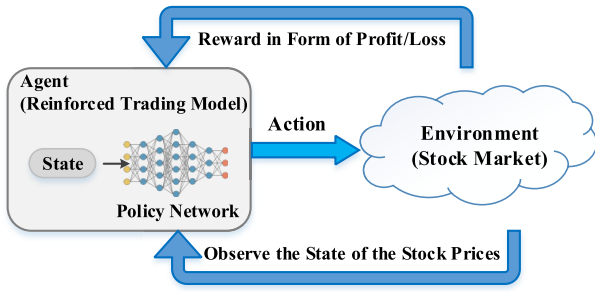


FIGURE 2. Markov decision process model.

on decisions guided by the strategy network. The market, in turn, provides profit/loss rewards in the form of the current stock price, which the agent utilizes to enhance the network through reinforcement learning. In our scenario, given the substantial trading volume of stocks in large companies, it is reasonable to assume that the behavior of individual traders during model training does not directly influence the price process. However, once the network has been trained on historical data and deployed for real-time online learning, it can effectively discern the impact of our trading behavior on the price process and subsequently adapt the optimal strategy accordingly. We define the state vector as $s_t = [p_t, s_t]$, where s_t represents other stochastic processes. Price and other states are jointly stochastic and may be influenced by the actions taken at time t , such that $s_{t+1} = f(S_t, x_t)$ is random. The objective is to maximize the anticipated overall profit from share sales, accounting for transaction fees, as indicated in Equation (1).

$$\operatorname{argmax}_{x_0, x_1, \dots, x_{T-1}} \mathbb{E} \left[\sum_{t=0}^{T-1} R(p_t, x_t) \right] \quad (1)$$

where $R(p_t, x_t)$ is the profit obtained from selling x_t shares at mid-price p_t . We work with the full state vector s_t from this point and s are various selected features.

B. PROBLEM FORMULATION

In this section, we will present the Proximal Policy Optimization (PPO) formulation of the optimal trade execution problem, detailing the states, actions, rewards, and algorithms of the reinforcement learning model utilized in our experiments. The execution time span is divided into T periods, with execution decisions made at the onset of each period. Trades are executed once per second for granularity.

1) STATES

The state space, denoted as S , encompasses the state of the Limit Order Book (LOB) at the onset of each period, along with any pertinent prior information from preceding periods. We adopt the standard assumption that given a state $s_t \in S$ and a trading action $x_t \in X$ at time t , the mapping (s_t, x_t) to s_{t+1} is Markovian. In homogeneous processes, time can be incorporated by allowing one component of the state space to represent time t itself. We explore various features in the

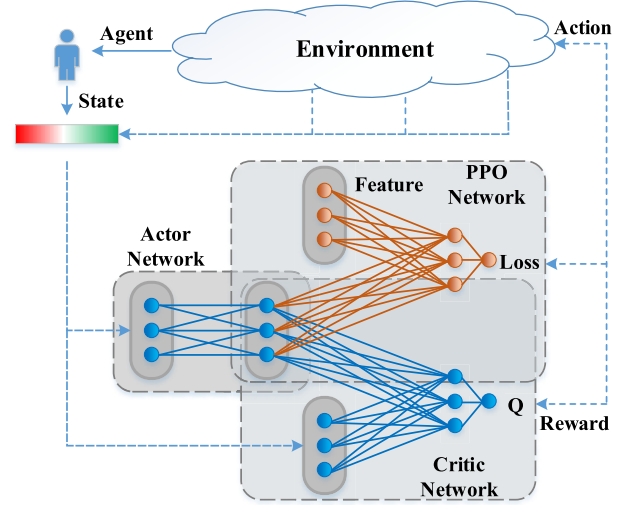


FIGURE 3. PPO architecture.

results section. However, two states play pivotal roles in the optimal strategy: (i) the current time (or elapsed time) t , and (ii) the remaining inventory q_t to be executed, both of which are consistently included in our state space.

2) ACTIONS

A policy uniquely maps a state $s \in A$ to an action $x \in X$. In the optimal execution problem, the sole action pertains to the quantity of shares to be sold via a market order. Naturally, the set of permissible actions is constrained such that $x_t \in [0, q_t]$ for all t . Additionally, we impose further limitations on the action space, restricting it to integer values (or fixed multiples of integer values for larger inventories). This simplifies the computational complexity required to determine the optimal action at each time step. Moreover, during each executable time step for actions, the total amount of shares sold is assumed to be evenly distributed over the time block on a second-by-second basis.

3) REWARDS

The reward equals the total reward over each period, and each period reward is made of rewards for trading each second which is shown as Equation (2).

$$R_t = \sum_{i=1}^{N_t} r_{t,i} \quad (2)$$

where R_t denotes the total reward for time period t , N_t represents the number of time steps within period t , and $r_{t,i}$ signifies the reward per second. Specifically, the reward per second can be formulated as Equation (3):

$$r_{t,i} = \frac{x_{t,i}}{N_t} - \alpha_i \left(\frac{x_{t,i}}{N_t} \right)^2 \quad (3)$$

where $x_{t,i}$ denotes the quantity of orders placed at time step i , and α_i serves as the hyperparameter for the quadratic penalty term, allowing for adjustment of trading costs and market

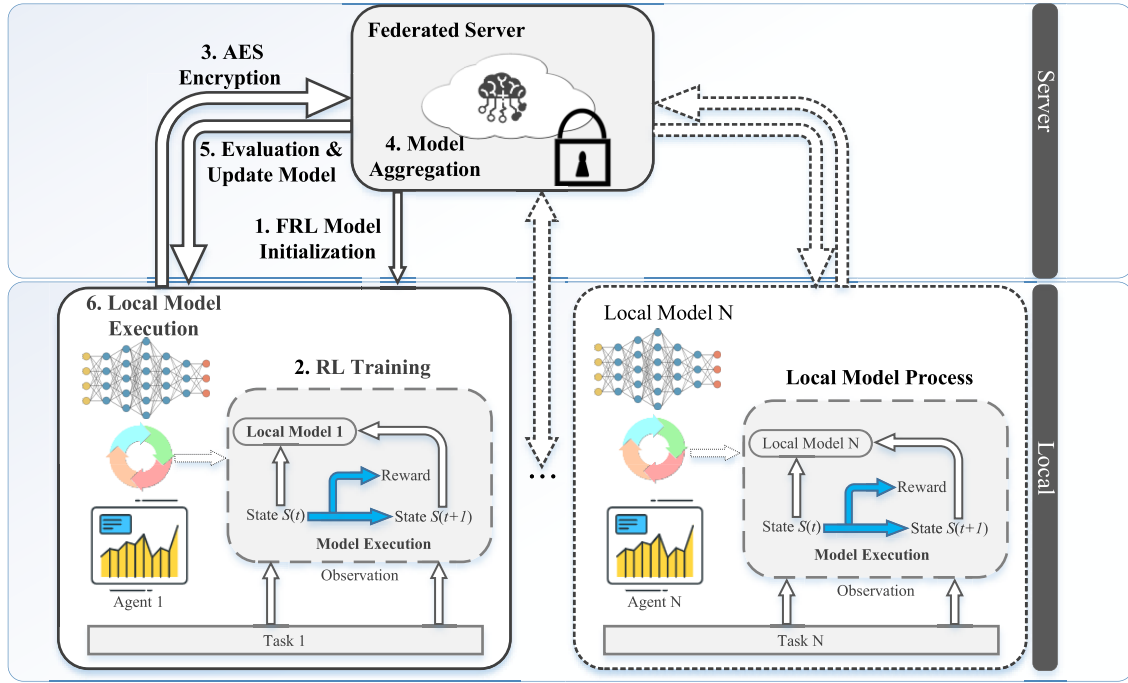


FIGURE 4. FPPO framework.

impact. Different participants can choose varying α_i values according to their risk preferences to customize the shape of the reward function.

The PPO algorithm begins by initializing the parameters of the policy and value networks, followed by collecting data through interactions between the agent and the environment. It then calculates an advantage estimate for each action, reflecting the difference between the expected and actual rewards. During the policy update phase, the algorithm computes the probability ratio between the new and old policies, constraining this ratio with a clip function to ensure that policy changes are not too drastic. Concurrently, the value network is updated based on TD errors to optimize the prediction of state values. This process iterates until the policy performance no longer improves or a set number of iterations is reached. The PPO algorithm is favored for its simple yet effective optimization mechanism, stable training process, and scalability in handling large-scale data and high-dimensional state spaces. This makes it a powerful tool in areas such as building trade execution systems. The goal of the PPO algorithm is to maximize the proximal strategy optimization objective, which is usually denoted as Equation (4):

$$\mathcal{L}(\theta) = \mathbb{E}_t \left[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t) \right] \quad (4)$$

where $r_t(\theta)$ denotes the probability ratio before and after the strategy update, and calculates the ratio of the probability of taking action a to state s under the new strategy $\pi_\theta(a|s)$ and the old strategy $\pi_{\theta_{\text{old}}}(a|s)$, as shown in Equation (5). $\hat{A}_t$ denotes the estimate of the dominance function, which

measures the number of states at state s by taking action a compared to the additional payoff that can be obtained by the average strategy. And ϵ denotes the clipping parameter. The clip function restricts the probability ratio $r_t(\theta)$ to the range $[1 - \epsilon, 1 + \epsilon]$ to prevent the performance degradation caused by too long update step, thus maintaining the stability of policy update and the balance between exploration and utilization. This approach makes the PPO algorithm show better performance and stability in practice.

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \quad (5)$$

The cumulative reward is computed as the summation of rewards across each time period, with the final equality derived from the conditions $q_t = 0$ and $q_t - q_{t-1} = x_t$. This is achieved through the application of a summation by parts formula. The overall reward is determined by the quantity sold in each time period, adjusted by a penalty proportional to the amount sold.

To evaluate the performance of their solution, two key financial metrics are utilized: the Average Profit Margin and the Total Trade Profit. These metrics provide a comprehensive reflection of the profitability of the trading strategy and the economic benefit of each trade. The calculation methods for these metrics are described as follows:

$$\text{Average Profit Margin} = \frac{\sum_{i=1}^N \text{Profit}_i}{N} \quad (6)$$

where N denotes the number of trades, and Profit_i denotes the profit of the i th trade. Profit is calculated as the net gain

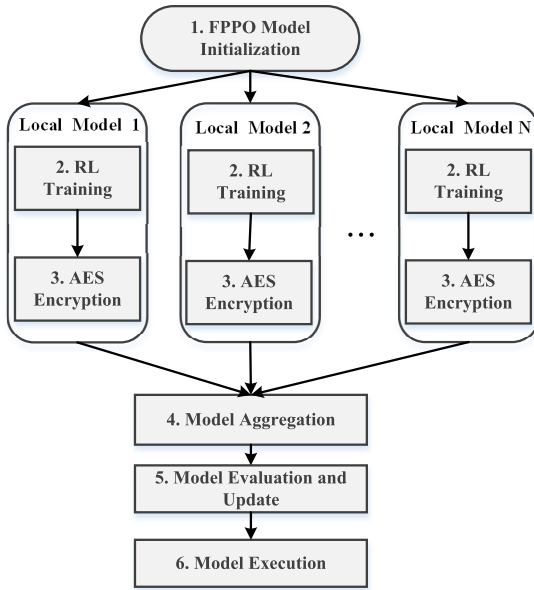


FIGURE 5. Flowchart of FPPO algorithm.

obtained from the i th trade after deducting all transaction costs, including but not limited to trading fees and slippage losses.

$$\text{Total Trade Profit} = \sum_{i=1}^N \text{Profit}_i \quad (7)$$

This metric sums up the profits from all trades, offering a holistic view of the overall profit generated within a certain period without considering the number of trades. It provides a direct view of the total profit produced by the strategy over a defined time frame.

In the computation of these metrics, it is ensured that the profits and losses of all trades are determined based on actual transaction prices and quantities, and all relevant transaction costs have been taken into account. Such an evaluation method not only offers an intuitive understanding of the strategy's performance but also provides data support for further strategy optimization.

C. PROPOSED FPPO SYSTEM ARCHITECTURE

In this section, the system architecture of FPPO will be introduced. The proposed FPPO system architecture, as depicted in Fig.4, integrates a federated learning approach with the PPO algorithm to create a privacy-preserving optimal trading execution framework. This architecture enables the distributed training of a global model across local agents while ensuring data privacy through encryption, resulting in an adaptive and efficient trading strategy.

The FPPO system architecture is comprised of several key components that facilitate a distributed learning environment. Initially, the global model is initialized on a federated server, which then distributes this model to various local agents. These agents are responsible for training and reinforcement learning processes using their respective local data. Once the

models have been trained at the local level, they are encrypted to ensure data privacy and securely transmitted back to the federated server for aggregation.

Algorithm 1 RL Training Algorithm

Input: Initialize global Agent parameters θ , Q-function parameters ϕ , empty replay buffer $\mathcal{D}$, num of Agents $\mathcal{N}$
 Set N Agents parameters θ_i, ϕ_i equal to global parameters θ, ϕ ;
for $Epochs=1$ to $\mathcal{M}$ **do**
 Initialize a random process $\mathcal{N}$ for action exploration;
 for $Agent=1$ to N **do**
 Receive initial observation state s_1 ;
 With probability ϵ select a random action a_t ,
 Otherwise select $a_t = \max_a Q^*(\phi(s_t), a; \theta)$;
 Execute action a_t and observe reward r_t and observe new state s_{t+1} ;
 Set $s_{t+1} = s_t, a_t, x_{t+1}$ and process $\phi_{t+1} = \phi_{s_{t+1}}$;
 Store transition $(\phi_t, a_t, r_t, \phi_{t+1}, done)$ in $\mathcal{D}$;
 Sample a mini-batch of transitions
 $(\phi_j, a_j, r_j, \phi_{j+1}, done)$ from $\mathcal{D}$;
 Set $y_i = \begin{cases} r_j, & \text{for terminal } \phi_{j+1} \\ r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta), & \text{for non terminal } \phi_{j+1} \end{cases}$
 Perform a gradient descent step on
 $(y_i - Q(\phi_j, a'; \theta))^2$
 if $Epochs \bmod AggPer = 0$ **then**
 θ encrypted by AES
 $\theta = \frac{1}{N} \sum_{i=1}^N \theta_i$
 end if
 end for
end for

D. FLOWCHART OF FPPO FRAMEWORK

To address the problem of optimal trade execution using federated reinforcement learning, we propose to utilize the FPPO algorithm. FPPO is a variant of the PPO algorithm that has been adapted for federated learning scenarios. The flowchart involved in our methodology are as shown in Fig. 5:

1. **FPPO Model Initialization:** The process begins by identifying a consortium of market participants, including financial institutions and trading agents, who consent to engage in the federated learning endeavor. Participant selection is based on criteria such as expertise, trading volume, and commitment to contribute to the federated learning initiative. The aggregator initializes a global model, which is then distributed to the local models of each participant, establishing an interconnected framework for collaborative learning.

2. **RL Training:** As delineated in Algorithm 1, the global parameters are initialized, followed by $\mathcal{M}$ batches of iterations. Each batch involves local exploration by the agents to

TABLE 1. Experimental configuration.

Experimental parameter	Experimental configuration
Hardware Configuration	Intel Core (TM) i9-10900K CPU @ 3.70 GHz, Quadro P2200
Software Version	NVIDIA GPU
Training Data	Pytorch 1.12.0
Testing Data	January 2010 Through December 2022
Mid Price Extraction	January 2023
Excluded Volatile Times	Full Limit Order Book Information
Initial Funding	9,30 a.m. to 11.30 a.m. and 1 p.m. to 3 p.m.
Replay Memory	10000
Discount Factor	500
Number of AES Key Bits	0.99
AES Encryption Rounds	128
AES Public Key	10
	hKJxZkuIyUx8YopN

identify optimal actions a_t , which are then stored in a shared repository $\mathcal{D}$. A mini-batch is then randomly selected from this repository for local model training using gradient-based optimization techniques, such as the FedAvg algorithm, which facilitates the averaging of model weights across agents.

At the end of each aggregation round, the locally updated models undergo an encrypted transfer to a global server. This server aggregates the local updates to produce an optimized global model, which is then shared back with all agents for the next round of training. This ensures that each agent's model benefits from the knowledge learned by others, while maintaining privacy through encryption.

The local models are trained using the FPPO algorithm, which combines the strengths of Proximal Policy Optimization (PPO) with federated learning. PPO is applied for local reinforcement learning (RL) policy optimization, and it is adjusted based on the feedback from local exploration. Federated learning ensures that the local training can occur without direct access to sensitive data, as data privacy is preserved through secure model aggregation and local data partitioning. Each participant partitions their local data into non-overlapping subsets to prevent data leakage while contributing to the federated learning process.

Moreover, during training, the models undergo periodic fine-tuning based on their local reward signals, which help agents adapt their policies. The FPPO algorithm encourages more efficient learning through a combination of regularized policy updates and robust exploration, allowing agents to improve their decision-making progressively while maintaining the integrity of privacy-preserving protocols.

3. AES Encryption: At regular intervals, participants aggregate their local policy updates. Post-training, the model parameters are encrypted using the AES algorithm and transmitted securely to the Federated Server for decryption, ensuring that the global model benefits from collective intelligence without direct data exposure.

4. Model Aggregation: A subset of clients is randomly selected from the $\mathcal{N}$ clients, and the global server receives the encrypted client model, decrypts it with the AES public key and performs the FedAvg algorithm [24] for model

aggregation as shown as follows:

$$\omega_g^{t+1} \leftarrow \sum_{k=1}^K \frac{n_k}{N} \omega_k^{t+1} \quad (8)$$

5. Evaluation and Update: The server distributes the aggregated, converged optimal trade execution model to the local clients in an encrypted form. The local models fine-tune the aggregated global model based on their own risk preferences, constructing a personalized optimal trade execution model for each local client according to the risk preference factor α . Specifically, for client k , the Euclidean distance between the global model ω_g and the local model ω_k is first measured to determine their similarity. Then, the global model is updated based on the local model improvement, adjusted by the risk preference factor α , as shown in Equation (9).

$$\omega_k = \frac{\exp(-\|\omega_g - \omega_k\|^2) \omega_k}{\sum_{i \neq k}^K \exp(-\|\omega_g^i - \omega_k^i\|^2)} + \alpha \omega_g \quad (9)$$

where $-\|\omega_g - \omega_k\|^2$ represents the Euclidean distance between the global model ω_g and the local model ω_k and α is the risk preference hyper-parameter. A higher value of α indicates a greater tolerance for risk, which can lead to potentially higher rewards.

6. Model Execution: Once the local model is trained and updated, it can be deployed for real-time trade execution in the market. The model continuously receives market data and generates optimal trade execution decisions based on the learned policies and strategies.

III. EXPERIMENTS AND STATISTICAL RESULTS

In this section, the experimental results of the proposed FPPO framework are presented and compared with existing algorithms, namely TWAP, AC, and DQN. To evaluate the performance of our approach, extensive simulations were conducted on multiple stocks and the outcomes were analyzed based on various performance metrics.

Firstly, the FPPO algorithm was tested against TWAP, AC, and DQN algorithms across a diverse set of stocks.

By utilizing raw secondary market micro-structure data and considering temporal correlation, informed trading decisions were aimed to be made. The superiority of our approach is intended to be demonstrated by comparing the average and median excess returns, profit-loss ratios, and probability of exceeding returns. Next, the performance of FPPO in trading large-cap stocks with a market capitalization greater than 30 billion USD was evaluated. These stocks are known for their stability and low volatility. The effectiveness of our FPPO framework in generating profitable trades while maintaining privacy for all parties involved was aimed to be assessed. Subsequently, the performance of FPPO in trading small-cap stocks with a market capitalization less than 10 billion USD was examined. These stocks are characterized by high volatility and potential for significant price fluctuations. The ability of our FPPO algorithm to adapt to market changes and generate optimal trading strategies was aimed to be investigated. Finally, the performance of FPPO when simultaneously trading multiple stocks was evaluated. This scenario aimed to simulate real-world trading conditions where an algorithm must make simultaneous decisions across different assets. The scalability and ability of our FPPO framework to handle complex trading environments effectively were aimed to be demonstrated.

In the context of optimal trading, the FPPO algorithm requires careful representation and preprocessing of input data. This involves extracting meaningful information through techniques like calculating technical indicators (e.g., moving averages, RSI) from raw market data. Subsequently, the extracted features undergo normalization to maintain a consistent scale, preventing any single feature from dominating the learning process. The preprocessed features are then combined to construct the state representation for the trading model, capturing current market conditions and guiding the reinforcement learning agent in making effective trading decisions.

In Table 1, it is noted that all tests and simulations are based on Windows 10 PC with Intel Core (TM) i9-10900K CPU @ 3.70 GHz and Quadro P2200 NVIDIA GPU. The neural network models are fully constructed with Pytorch version 1.12.0. The stock data used for model training covers the period from January 2010 to December 2022, while January 2023 stock data was utilized for comprehensive testing. The data is sourced from the website of China Securities Journal [25]. The mid price at the end of each second is extracted using the full limit order book information, and the most volatile times of the day are excluded by analyzing the hours 9.30 a.m. to 11.30 a.m. and 1 p.m. to 3 p.m. separately. Main-target network update after $p = 10$ iterations, and a replay memory of size $N = 10,000$. We fix our discount factor $\gamma = 0.99$ for all tests. The number of AES encryption bits in this experiment is 128. The strong public key is generated by a secure random number generator, and then the generated key is securely stored in an encrypted hardware module or in a software solution that uses the encryption services provided by the operating system to prevent unauthorized

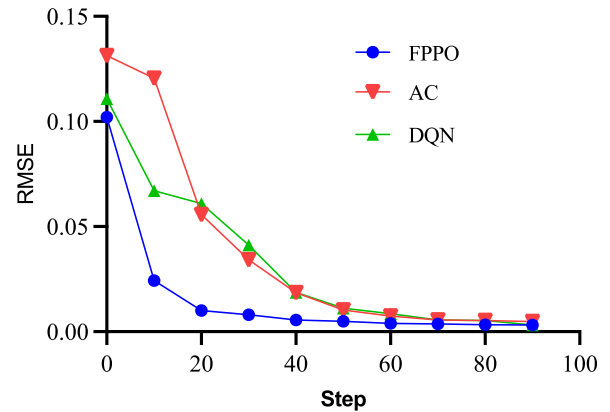


FIGURE 6. Comparison of RMSE between algorithms.

access. When the model is updated, the model parameters are encrypted using the AES algorithm and the generated key to ensure security during transmission and storage. The encrypted parameters are sent over secure communication channels to where they are needed, such as a central server or other nodes in a federated learning scenario. Finally the receiver decrypts the encrypted parameters using the same key to recover the original model parameters, which can then be applied for local model updating. Once a network model is trained on data, we iterate through all the time steps, inventory levels and price levels and compute the optimal actions traced out in every state.

A. COMPARISON OF RMSE ALGORITHMS

Fig. 6 illustrates the Root Mean Square Error (RMSE) comparison among different algorithms, including FPPO, AC, DQN. The x-axis represents the steps or iterations in the simulation or testing phase, while the y-axis depicts the RMSE values.

In the graph, the FPPO algorithm demonstrates a consistently lower RMSE across the steps when compared to the AC and DQN algorithms. This indicates that the FPPO algorithm has a smaller error magnitude in its predictions or decisions, suggesting higher accuracy and better performance in the context of the study. The RMSE for FPPO is notably stable and lower than the other algorithms, particularly in the latter half of the steps, implying that the model's predictions are more reliable and precise.

For the AC algorithm, the RMSE fluctuates but generally remains higher than that of FPPO, indicating less accuracy in its trading execution strategy. The DQN algorithm also shows higher RMSE values, suggesting that it may be more prone to errors or have less precise predictions compared to FPPO. This performance advantage is crucial in stock trading execution, where precision in predicting market movements can lead to more profitable and efficient trading strategies.

B. ENCRYPTION TIME COMPARISON

To address the computational and time costs associated with encryption in the FPPO framework, we conducted a

series of experiments to quantify the impact of encryption on model training and update processes. Our experiments were conducted on a system with PyTorch 1.12.0 and the AES encryption algorithm with a 128-bit key length. The number of encryption rounds is 10, and the public key is hKJxZkuIyUx8YopN.

The encryption and decryption times for model parameters were measured before and after each training episode. The results, as presented in Table 2, indicate that the encryption process introduces an average overhead of 2.4% in terms of training time per episode, with decryption contributing an additional 5.7%.

The experimental data demonstrates that while encryption does impose some computational and time costs, these are manageable. It is suggested that further optimization is possible through the implementation of more efficient encryption algorithms or the utilization of hardware acceleration.

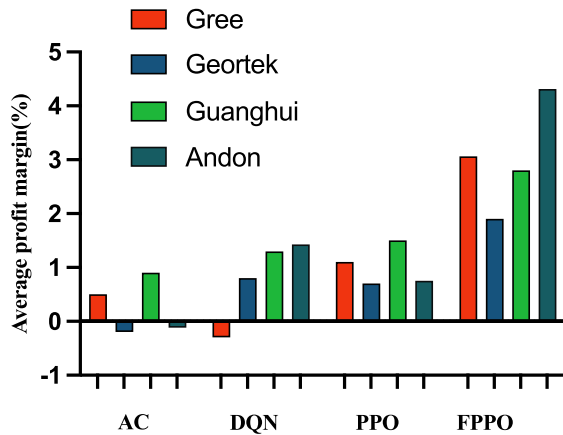


FIGURE 7. Average margin in differences between algorithms.

TABLE 2. Impact of encryption on training time.

Aspect	Average Time per Episode (s)	Overhead (%)
Training (unencrypted)	122	0
Encryption	3	2.4
Decryption	7	5.7
Total Training (encrypted)	132	-

C. SINGLE STOCK MODEL

To reduce the correlation among stocks, the stock selection includes randomly selected stocks from different industries, such as Gree Electric Appliances, Guanghui Energy, Sailisi with a market value exceeding 30 billion USD, Huagong Science and Technology, Zhangjiang High-tech, Samsung Medical, Geer with a market value between 10 billion USD and 30 billion USD, as well as Chuanda Zhisheng and other stocks with a market value less than 10 billion USD.

Table 3 shows various comparative statistics of the learned strategies using average profit margin for a collection of assets. The average profitability of four companies, namely

TABLE 3. Model stock average profit margin.

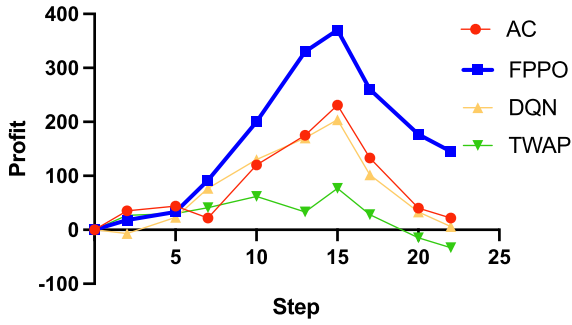
Model Name	Average Profit Margin			
	Gree	Goertek	Guanghui	Andon
TWAP	0.00	0.00	0.00	0.00
AC	0.57	-0.23	0.91	-0.12
DQN	-0.33	0.81	1.33	1.43
PPO	1.17	0.72	1.55	0.75
FPPO	3.06	1.90	2.80	4.31

Gree, Goertek, Guanghui, and Andon, has been extracted and presented in the table. The results indicate that the FPPO algorithm achieves a profit margin of over 3%, which is significantly higher than the AC, DQN, and PPO algorithms, which have a profit margin of less than 1.5%. It is worth noting that the model used in this study employs a federal average of three agents. Furthermore, it is observed that both the AC and DQN algorithms also exhibit some losses, highlighting the effectiveness of the FPPO algorithm in achieving low risk and high profitability.

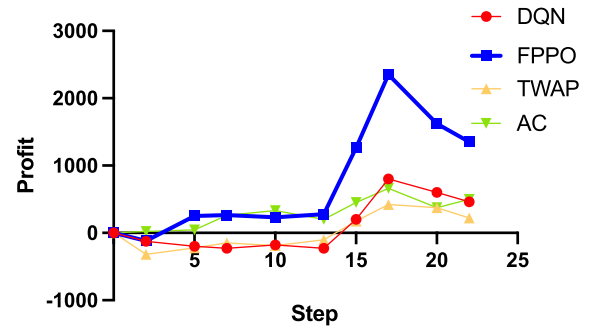
The average profitability is depicted in Fig. 7. It is indicated that the FPPO algorithm outperforms the other algorithms in terms of profitability for Gree, with a profitability rate exceeding 3%. Goertek also achieves a profitability rate of 1.9%, while Guanghui has a profitability rate of 2.8%. In contrast, the AC algorithm results in varying levels of losses for Goertek and Gree, respectively. The PPO algorithm performs worse than the FPPO algorithm but better than the other two algorithms.

In addition, we conducted a comparative analysis of FPPO and other algorithms in terms of their profitability in the stock market, specifically focusing on China Merchants Bank. As depicted in Fig. 8(a), the ordinary algorithm refers to the average results obtained from TWAP, AC, and DQN algorithms. It is evident that the FPPO algorithm outperforms the ordinary algorithm in terms of overall profitability. The unit of profitability in this study is the profit per 10,000 yuan principal. It can be observed that during periods when the market is not favorable (15-25 STEPS), all the profits generated by the ordinary algorithm are either lost or even result in a loss. This highlights the exceptional risk control ability of the FPPO algorithm.

Some of the companies listed previously have a larger market capitalization (greater than 20 billion USD), resulting in smaller overall stock price fluctuations. To further test the performance of the FPPO algorithm, we selected Xujiahui Business, a small-cap company, as shown in Fig. 8(b). Due to its small market capitalization, it can be observed that the stock's volatility is high, exceeding 20%. However, despite this, the FPPO algorithm achieved a maximum profit of 24%, with retracement not exceeding 5%. In contrast, the ordinary algorithm suffered a loss in the early stages and only reached a profit margin of about 8% when the market improved later on. This result demonstrates the applicability of the proposed



(a) Algorithms In Merchants Bank Model Average Margins

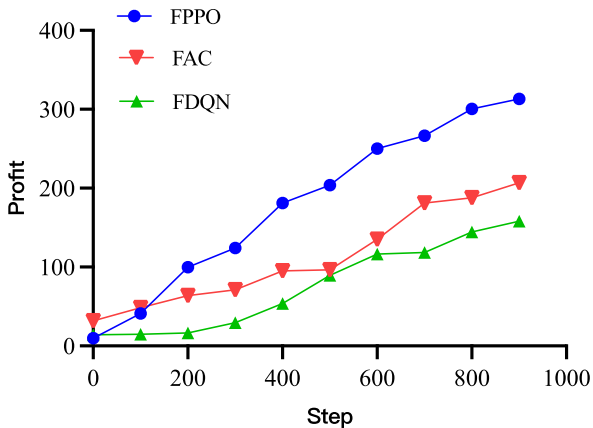


(b) Algorithms In Xujiashui Commercial Model Average Margins

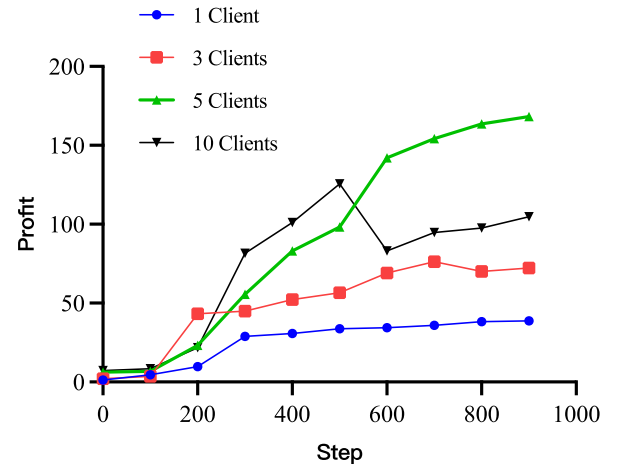
FIGURE 8. Average margins of single stock prediction model.

model in situations of small market capitalization and high volatility, highlighting its stability.

To verify the performance of the algorithm, FPPO was also compared experimentally with other federated reinforcement learning algorithms, such as FedDQN and FedAC. Fig. 9 illustrates the variation of the reward function values of these algorithms with the number of training iterations. As shown in the figure, the reward function of the FedPPO algorithm surpasses that of the FedAC and FedDQN algorithms after 100 iterations, eventually reaching approximately 300. In contrast, the reward functions of the FedAC and FedDQN algorithms reach around 200 and 150, respectively, while showing an overall upward trend. This indicates that the FedPPO algorithm can achieve better model performance under certain fixed parameters.

**FIGURE 9.** Comparison of federated reinforcement learning algorithms.

In constructing a federated multi-agent optimal trade execution model, the number of clients is also a crucial factor to consider. Experiments were conducted to explore the impact of client numbers on system performance by setting the number of clients participating in the federated reinforcement learning model to $K = 1, 3, 5, 10$. As shown in Fig. 10, during the initial training phase, the system with 10 clients achieved a relatively high reward function value. However, as the number of iterations increased, the reward

**FIGURE 10.** Comparison of different numbers of clients.

function value for 10 clients showed a noticeable decline. This indicates that having more clients is not always better for the optimal trade execution problem; instead, a moderate number of clients is preferable. Specifically, the system with 5 clients achieved a stable maximum reward function value of approximately 160. In comparison, the system with 3 clients stabilized at around 60, which is higher than the approximately 40 reward function value achieved by a single client. This further demonstrates that the federated reinforcement learning model outperforms single-client models in constructing an optimal trade execution framework.

D. MULTI-STOCK MODEL

In order to evaluate the effectiveness of the proposed framework, we applied it to multiple stocks for simultaneous trading and compared the results with those obtained from single stock trading. As shown in Fig. 11(a), three different types of stocks, namely Gree, Goertek, and Guanghui, were selected for simultaneous trading. The average return of Goertek was found to be the highest, reaching more than 26%, while the average returns of Gree and Guanghui were both within the range of 10%. The FPPO algorithm was able to maximize the profit margin up to 20%, which resulted in a

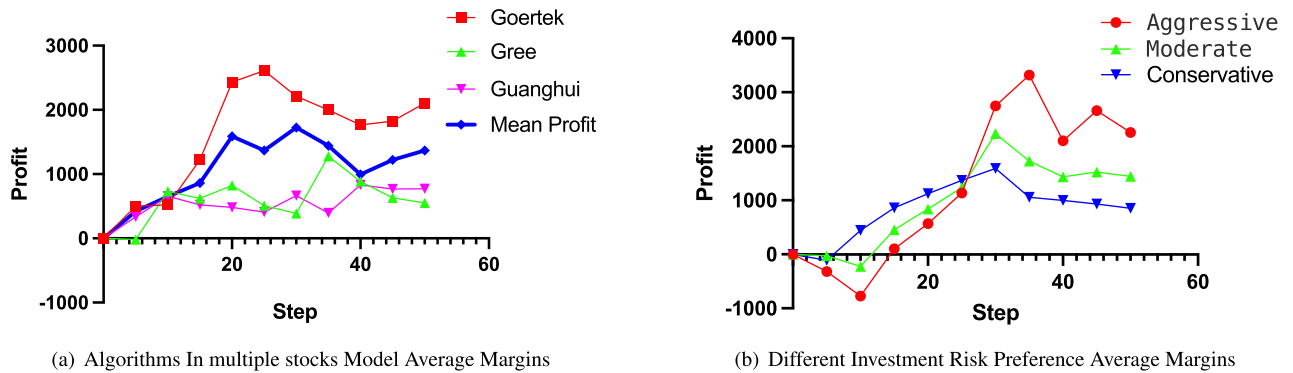


FIGURE 11. Average margins of multi-stock prediction model.

higher position on Goertek. This highlights the effectiveness of our approach in choosing the strategy that allows for a larger return.

Furthermore, the final model profit also reached more than 15%, reflecting the reasonableness of our model in setting the reward function to allow for the most profitable stocks to receive more positions and to reduce the positions in stocks that are not sufficiently profitable. This demonstrates the good profit-taking ability of the proposed model. Overall, our findings suggest that the FPPO algorithm is an effective tool for stock trading and can provide significant benefits to investors seeking to maximize their profits.

To assess the impact of risk appetite on our model, stock returns for three distinct stocks (Gree, Gorertek, and Guanghui) were forecasted using the FPPO algorithm while adjusting the CLIP to reflect varying levels of risk aversion. As shown in Fig. 11(b), it was found that substantial fluctuations in returns, ranging from -7% to 33% , can be expected by the aggressive investor, who is characterized by a high tolerance for risk. This approach may yield higher potential rewards but also entails greater volatility. Conversely, a more modest range of 3% to 14% can be anticipated by conservative investors, who prioritize stability over potential gains. Moderate investors, seeking a balance between risk and reward, can expect returns within the range of -2% to 22% . The probability of simultaneous excess returns exceeding 70% . In summary, trading strategies are effectively tailored to individual risk preferences by our FPPO model, proving its efficacy across diverse investor profiles.

A comprehensive comparison with commonly used industry algorithms was conducted to evaluate the practical applicability of the proposed algorithm. This step was essential for several reasons: Benchmarking against industry standards allows for an objective assessment of the algorithm's performance in terms of execution efficiency, risk management, and adaptability to market conditions. Practical validation ensures that the solution can effectively address real-world challenges such as market impact, transaction costs, and regulatory constraints. Additionally, identifying potential advantages, such as data privacy preservation while maintaining high

performance, can enhance the algorithm's credibility and facilitate its adoption in real-world trading environments.

While the FPPO model demonstrates strong adaptability and efficacy in tailoring trading strategies to varying risk preferences, several challenges and limitations could arise in real-world trading environments. First, the model's reliance on accurate and timely data is critical, yet real-world data may contain noise or suffer from delays that could impact decision-making. Second, computational efficiency is a concern, especially when scaling to a larger number of stocks or institutions, as federated learning and reinforcement learning both demand substantial resources for training and updating. Third, privacy regulations and data-sharing restrictions across different jurisdictions could complicate the implementation of federated learning frameworks. Additionally, the model's sensitivity to market conditions may require frequent retraining or fine-tuning to remain effective in dynamic financial environments. Addressing these challenges would require ongoing efforts to optimize computational efficiency, enhance data robustness, and ensure regulatory compliance, thus paving the way for the broader adoption of such advanced trading frameworks.

IV. CONCLUSION

In this paper, a framework based on the FPPO algorithm is proposed to address the optimal execution problem. This problem is formulated as a joint reinforcement learning problem involving multiple agents. To ensure privacy security, modifications are made to the PPO method. Specifically, federated learning is adopted to enable multiple agents to collaboratively construct an optimal trade execution model securely. Moreover, the AES algorithm is employed to encrypt trade and model data. Experimental results clearly showcase the superior performance of proposed approach, surpassing TWAP, AC, and DNQ algorithms when evaluated across many kinds of stocks. Furthermore, the model's performance is validated through simultaneous trading of multiple stocks. Additionally, the risk preference payoff function can be utilized to control risk, allowing different financial institutions to select their preferred model.

Future work could explore advanced privacy-preserving techniques such as differential privacy and homomorphic encryption to enhance data security further. Expanding the framework's scalability to larger markets and dynamic conditions, along with integrating advanced risk management tools like value-at-risk, could improve its adaptability. Incorporating techniques like meta-learning or transfer learning may enhance the model's generalization across diverse stocks and markets. Real-world implementation and testing in collaboration with financial institutions would provide valuable insights, while considering environmental and ethical factors could align the framework with sustainable finance practices. These directions aim to refine and expand the framework's applicability in real-world trading scenarios.

ACKNOWLEDGMENT

(Haogang Feng and Yue Wang are co-first authors.)

REFERENCES

- [1] M. F. Anaghi and Y. Norouzi, "A model for stock price forecasting based on ARMA systems," in *Proc. 2nd Int. Conf. Adv. Comput. Tools Eng. Appl. (ACTEA)*, Dec. 2012, pp. 265–268.
- [2] A. Tsantekidis, N. Passalis, A. Tefas, J. Kannianen, M. Gabbouj, and A. Iosifidis, "Forecasting stock prices from the limit order book using convolutional neural networks," in *Proc. IEEE 19th Conf. Bus. Informat. (CBI)*, vol. 1, Jul. 2017, pp. 7–12.
- [3] R. A. Kamble, "Short and long term stock trend prediction using decision tree," in *Proc. Int. Conf. Intell. Comput. Control Syst. (ICICCS)*, Jun. 2017, pp. 1371–1375.
- [4] O. Guéant and C.-A. Lehalle, "General intensity shapes in optimal liquidation," 2012, *arXiv:1204.0148*.
- [5] B. Hambly, R. Xu, and H. Yang, "Recent advances in reinforcement learning in finance," 2021, *arXiv:2112.04553*.
- [6] J. Clifton and E. B. Laber, "Q-learning: Theory and applications," *Annu. Rev. Statist. Appl.*, vol. 7, no. 1, pp. 279–301, Mar. 2020.
- [7] G. Jeong and H. Y. Kim, "Improving financial trading decisions using deep Q-learning: Predicting the number of shares, action strategies, and transfer learning," *Expert Syst. Appl.*, vol. 117, pp. 125–138, Mar. 2019.
- [8] B. Ning, F. H. T. Lin, and S. Jaimungal, "Double deep Q-learning for optimal execution," *Appl. Math. Finance*, vol. 28, no. 4, pp. 361–380, Jul. 2021.
- [9] S. Lin and P. A. Beling, "A deep reinforcement learning framework for optimal trade execution," in *Proc. Joint Eur. Conf. Mach. Learn. Knowl. Discovery Databases*, Jan. 2021, pp. 223–240.
- [10] D. Chen, Y. Zhu, M. Liu, and J. Li, "Cost-efficient reinforcement learning for optimal trade execution on dynamic market environment," in *Proc. 3rd ACM Int. Conf. AI Finance*, New York, NY, USA, Nov. 2022, pp. 386–393.
- [11] S. Kim, J. Kim, H. K. Sul, and Y. Hong, "An adaptive dual-level reinforcement learning approach for optimal trade execution," *Expert Syst. Appl.*, vol. 252, Oct. 2024, Art. no. 124263.
- [12] C. C. Moallemi and M. Wang, "A reinforcement learning approach to optimal execution," *Quant. Finance*, vol. 22, no. 6, pp. 1051–1069, Jun. 2022.
- [13] Y. Ansari, S. Yasmin, S. Naz, H. Zaffar, Z. Ali, J. Moon, and S. Rho, "A deep reinforcement learning-based decision support system for automated stock market trading," *IEEE Access*, vol. 10, pp. 127469–127501, 2022.
- [14] K. Bisht and A. Kumar, "Deep reinforcement learning based multi-objective systems for financial trading," in *Proc. 5th IEEE Int. Conf. Recent Adv. Innov. Eng. (ICRAIE)*, Dec. 2020, pp. 1–6.
- [15] D. Klijn and A. E. Eiben, "A coevolutionary approach to deep multi-agent reinforcement learning," 2021, *arXiv:2104.05610*.
- [16] W. Zhao, J. P. Queralta, and T. Westerlund, "Sim-to-real transfer in deep reinforcement learning for robotics: A survey," in *Proc. IEEE Symp. Ser. Comput. Intell. (SSCI)*, Dec. 2020, pp. 737–744.
- [17] J. Liu, J. Lou, L. Xiong, and X. Meng, "Personalized differentially private federated learning without exposing privacy budgets," in *Proc. 32nd ACM Int. Conf. Inf. Knowl. Manage.*, New York, NY, USA, Oct. 2023, pp. 4140–4144.
- [18] T. Awosika, R. M. Shukla, and B. Pranggono, "Transparency and privacy: The role of explainable AI and federated learning in financial fraud detection," 2023, *arXiv:2312.13334*.
- [19] V. S. Parekh, S. Lai, V. Braverman, J. Leal, S. Rowe, J. J. Pillai, and M. A. Jacobs, "Cross-domain federated learning in medical imaging," 2021, *arXiv:2112.10001*.
- [20] J. Qi, Q. Zhou, L. Lei, and K. Zheng, "Federated reinforcement learning: Techniques, applications, and open challenges," 2021, *arXiv:2108.11887*.
- [21] S. Lin and P. A. Beling, "An end-to-end optimal trade execution framework based on proximal policy optimization," in *Proc. 29th Int. Joint Conf. Artif. Intell.*, C. Bessiere, Ed., Jul. 2020, pp. 4548–4554.
- [22] S. Sharma and S. Bhadula, "Secure federated learning for intelligent industry 4.0 IoT enabled self skin care application system," in *Proc. 2nd Int. Conf. Appl. Artif. Intell. Comput. (ICAAIC)*, May 2023, pp. 1164–1170.
- [23] X. Liang, Y. Liu, T. Chen, M. Liu, and Q. Yang, "Federated transfer reinforcement learning for autonomous driving," 2019, *arXiv:1910.06001*.
- [24] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. Y. Arcas, "Communication-efficient learning of deep networks from decentralized data," 2016, *arXiv:1602.05629*.
- [25] Cninfo inference. (2023). *Cninfo*. Accessed: Jan. 15, 2023. [Online]. Available: <http://www.cninfo.com.cn/new/index/>



HAOGANG FENG was born in Henan, China, in 1997. He received the B.Sc. degree in electronics engineering from Shenzhen University, Shenzhen, China, in 2019, where he is currently pursuing the Ph.D. degree in electronics and electrical engineering. His current research interests include field-programmable gate array (FPGA), joint algorithm and hardware code-sign, and application-specified integrated circuit (ASIC) implementations for next-generation channel coding systems.



YUE WANG was born in Shanxi, China, in 1998. He received the Bachelor of Science degree in software engineering from Taiyuan University of Technology, in 2020. He is currently pursuing the Master of Science degree in electrical and electronic engineering with Shenzhen University. For example, he applies federated reinforcement learning to realize secure distributed machine learning in several privacy-sensitive domains. In addition, his focus on edge computing algorithms reflects his awareness of the critical role these algorithms play in optimizing data processing for applications, such as the Internet of Things (IoT). His research interests include federated reinforcement learning and edge computing algorithms.



SHIDA ZHONG received the B.Sc. degree in electronics engineering from Shenzhen University, Shenzhen, China, in 2008, and the M.Sc. and Ph.D. degrees in electronics and electrical engineering from the University of Southampton, Southampton, U.K., in 2009 and 2013, respectively. He is currently an Associate Professor with the College of Electronics and Information Engineering, Shenzhen University. His current research interests include low-power IC design, antenna selection for massive MIMO, FPGA and ASIC implementations for next-generation channel coding systems, joint algorithm, and hardware codesign for edge computing.



TAO YUAN (Member, IEEE) received the B.E. degree in electronic engineering and the M.E. degree in signal and information processing from Xidian University, Xi'an, Shaanxi, China, in 1999 and 2003, respectively, and the Ph.D. degree in electrical and computer engineering from the National University of Singapore, Singapore, in 2009. He has been a Distinguished Professor with the College of Electronics and Information Engineering, Shenzhen University, Shenzhen, Guangdong, China, since 2016. His current research interests include the design and implementation of novel RF frontend chips/modules and integrated devices/antennas/circuits for 5G/6G applications.



ZHI QUAN (Senior Member, IEEE) received the B.E. degree in communications engineering from Beijing University of Posts and Telecommunications (BUPT), China, in 1999, and the Ph.D. degree (Hons.) in electrical engineering from the University of California, Los Angeles (UCLA), in 2009. He was a Senior System Engineer with Qualcomm Research Center (QRC), Qualcomm Inc., San Diego, CA, USA, from 2008 to 2012, and as an RF System Architect with Apple Inc., Cupertino, CA, from 2012 to 2015. He is currently a Distinguished Professor with the College of Electronic and Information Engineering, Shenzhen University, China. He has been granted more than 40 patents and published more than 70 articles in wireless communications and signal processing with more than 5000 citations from Google Scholar. His current research interests include wireless communication systems, RF system calibration and measurement, data-driven signal processing, and machine learning. He was a recipient of the UCLA Outstanding Ph.D. Award, in 2009, the IEEE Signal Processing Society Best Paper Award, in 2012, China National Excellent Young Scientist Foundation, in 2016, and the First Prize Technology Innovation Award by China Institute of Communications, in 2020.

...